

Virtual Karaoke Implementation

Final Report



Prepared by **Vinny Manara, Jinghui Pang, and Svastik Patel**

For use in **CS440** at the **University of Illinois Chicago**

Table of Contents

List of Figures.....	4
List of Tables.....	5
I Project Description.....	6
1 Project Overview.....	6
2 Project Domain.....	6
3 Relationship to Other Documents.....	6
4 Naming Conventions and Definitions.....	7
4a Definitions of Key Terms.....	7
4b UML and Other Notation Used in This Document.....	7
4c Data Dictionary for Any Included Models.....	7
II Project Deliverables.....	8
5 First Release.....	8
6 Second Release.....	8
7 Comparison with Original Project Design Document.....	9
III Testing.....	9
8 Items to be Tested.....	9
8.1 Room Component (React + WebRTC + Socket.io).....	9
8.2 Node.js Signaling Server (app.js).....	9
8.3 FastAPI Song Service (songs_api.py).....	10
8.4 Integrated System Behavior.....	10
9 Test Specifications.....	10
9.1 Room Join & WebRTC Connection.....	10
9.2 Synchronized Video Playback.....	10
9.3 Queue Update Propagation.....	11
9.4 Song Search & API Response.....	11
9.5 YouTube Stream Extraction.....	12
9.6 Mic/Camera Toggle Behavior.....	12
10 Test Results.....	13
10.1 Room Join & WebRTC Connection.....	13
10.2 Synchronized Video Playback.....	13
10.3 Queue Update Propagation.....	14
10.4 Song Search & API Response.....	14
10.5 YouTube Stream Extraction.....	14
10.6 Mic/Camera Toggle Behavior.....	15

11 Regression Testing.....	15
11.1 Regression Test R1 — Playback Synchronization.....	15
11.2 Regression Test R2 — Queue Update Consistency.....	16
11.3 Regression Test R3 — YouTube Stream Extraction.....	16
IV Inspection.....	16
12 Items to be Inspected.....	16
12.1 Room Component (React + WebRTC + Socket.io).....	16
12.2 Node.js Signaling Server (app.js).....	17
12.3 FastAPI Song Service (songs_api.py).....	17
13 Inspection Procedures.....	17
13.1 Preparation.....	17
13.2 Inspection Meetings.....	17
13.3 Follow-Up.....	18
14 Inspection Results.....	18
14.1 Room Component (React + WebRTC + Socket.io).....	18
14.2 Node.js Signaling Server (app.js).....	18
14.3 FastAPI Song Service (songs_api.py).....	19
V Recommendations and Conclusions.....	20
VI Project Issues.....	20
15 Open Issues.....	20
16 Waiting Room.....	21
17 Ideas for Solutions.....	22
18 project Retrospective.....	22
VI Glossary.....	22
VI References / Bibliography.....	23
VI Index.....	23

List of Figures

List of Tables

I Project Description

1 Project Overview

Virtual Karaoke is a web-based application that enables users to participate in live karaoke sessions together over the internet. Users can create or join virtual rooms where they interact through real-time video, audio, and chat logs while taking turns singing their favorite songs. The system supports a shared song queue, synchronized soundtrack playback, and a searchable library of songs, allowing participants to preview, add, and perform tracks in a coordinated manner. Using WebRTC for peer-to-peer streaming and Socket.io for real-time communication, the application recreates the feel of in-person karaoke in a smooth, accessible, and user-friendly online environment.

2 Project Domain

The domain of this project is focused on the global karaoke entertainment industry, which traditionally operates through private karaoke rooms and public karaoke bars where groups gather to sing with on-screen lyrics, microphones, and shared song systems. However, the COVID-19 pandemic made in-person karaoke inaccessible, forcing many users to rely on general purpose meeting software that fails to recreate the same immersive social nature of karaoke. At the same time, broader trends in interactive entertainment have shifted toward online experiences that require minimal physical presence. Virtual Karaoke addresses these changes by transitioning karaoke into a fully digital environment, allowing users to participate from any device while still experiencing the core elements of singing, listening, and social interaction. By supporting real-time audio and video streaming, synchronized soundtracks, and shared room features the system fits squarely within the domain of remote entertainment technologies designed to replicate traditionally in-person activities in an online environment.

3 Relationship to Other Documents

This implementation project report builds upon several foundational documents that guided the development of Virtual Karaoke. The design project description, Virtual Karaoke Final Report, developed by the design team, established the initial problem, user background, business motivations, goals, and the broader domain in which the system operates. Also defining the current situation, competing products, stakeholders, mandated constraints, and the full scope of the work, providing the conceptual framework that shaped the direction of this coding phase. In addition, the design team's scenarios, use cases, requirements, and architectural outlines informed the structure of the system implemented here, including the room-based interaction model, audio/video streaming expectations, and song playback workflow. The Scenario 1 and Scenario 2 documents created for this implementation further refine the functional scope by specifying the features delivered in each release, such as room creation, chat, song search, queue management, synchronized playback, and

multi-user video interaction. This serves as the basis for the requirements, design decisions, and testing procedures presented in this report. These documents form a cohesive chain from the origin design intent to the final implementation prototype, all referenced in the bibliography.

4 Naming Conventions and Definitions

4a Definitions of Key Terms

Room: a virtual, video call-like session where have unique randomized 6-character roomId with 15 people limitation currently.

webRTC: web Real Time Communication, the protocol used to establish direct connection between users in a room for video and audio.

Performer: The user is actively singing, with highlighted indication and could see upvotes from other users in the room.

Queue: The shared playlist across all users in the room that could let user get ready to perform and adjust.

Signaling Server: The backend of node.js utilizes express and socket.io to deal with real-time events (webRTC communicating, queue updating, chat messaging).

ICE candidate: Network routing information that could be passed between clients between signaling servers to communicate with webRTC.

4b UML and Other Notation Used in This Document

Due to the nature of the real-time connection of virtual karaoke, the following are more focused on our components.

Solid arrow with filled head: denote synchronous HTTP/REST API calls.

Solid arrow with empty head: denote asynchronous HTTP/REST API calls.

dashed arrow with empty head: To indicate the data returns from API calls or database fetching.

Stereotype (<< >>): Used for diagrams to distinguish between different technology stacks used. (<<React>> used for the client UI, <<Node.js/Socket.io>> used for the signaling server).

4c Data Dictionary for Any Included Models

Song database (SQLite):

- id: INTEGER PRIMARY KEY AUTOINCREMENT

- title: TEXT NOT NULL
- duration: REAL

Room object:

- roomId: String
- users: Array of User Objects { socketId: String, username: String } (Maximum 15 users per room)
- messages: Array of Chat Objects { username: String, text: String, id: String }
- queue: Array of Song Objects { id: String, title: String, artist: String, previewUrl: String, addedBy: String }
- performer: Performer Object { socketId: String, username: String, upvotes: Integer, voterSocketIds: Array of Strings }
- nowPlaying: Song Object

II Project Deliverables

Over the course of this semester, our team produced a fully functional prototype of the Virtual Karaoke system, implemented as a web application using React, Node.js with Socket.io, FASTAPI, SQLite, and WebRTC. The deliverables include two incremental releases aligned with Scenario 1 and Scenario 2. The first release focused on establishing the core room-based interaction model, including room creation, room entry, chat functionality, and verification of camera and microphone access. The second release expanded the system to support a searchable song library, a shared room-wide queue, synchronized soundtrack playback, multi-user streaming, and interactive features such as upvoting the current performer..

5 First Release

The first release of the Virtual Karaoke system was implemented on the 2nd of February, 2026, and completed on the 27th of February, 2026. This implementation delivered the foundation features described in Scenario 1. In this release, users could create or join a room through the web interface, with the system generating unique room identifiers and validating room existence. Once inside a room, users had access to a functioning chat panel and could test their camera and microphone using browser media APIs. The system displayed live video from the user's webcam, confirmed microphone activity through a volume indicator, and allowed users to leave the room at any time. This release also established the initial Socket.io signaling structure and room state management on the server, laying the groundwork for real-time communication and future WebRTC integration. At this stage, no song playback, queueing, or multi-user video streaming was included; the focus was on ensuring reliable room creation, device access, and basic interaction.

6 Second Release

The first release of the Virtual Karaoke system was implemented on the 2nd of March, 2026, and completed on the 3rd of April, 2026. This implementation

delivered the full functionality described in Scenario 2 and transformed the prototype into a complete multi-user karaoke environment. This release introduced a searchable song library backed by a FastAPI service and SQLite database. Users could preview songs, add them to a shared room queue, and start playback for all participants simultaneously. The system supported synchronized lyric video playback, automatic highlighting of the active performer, and an upvote feature that allowed participants to react to performances in real time. WebRTC integration enabled multi-user video streaming, with dynamic video grids that updated as users joined, left, or toggled their cameras. Playback synchronization was maintained through Socket.io events, ensuring that play, pause, and seek actions were mirrored across all clients. This release completed the core functionality required for a realistic online karaoke experience.

7 Comparison with Original Project Design Document

Database: the original design mandate using MySQL for managing user data and song catalog. However, the current design uses SQLite to trigger the backend easily without using a server database.

Language: The original design recommended C or C++ for more memory-efficient to deal with heavy video streaming. Our implementation instead used React for client, python for song service and queueing.

Song Library: The original design suggested directly integrating with platforms like Spotify and Apple Music for a rich database of the music. Our prototype implementation bypassed complex authentication and licensing process used yt-dlp to load directly from YouTube video.

Room: The original design specified that each karaoke room should handle at least 50 people. While our implementation only hold up to 15 with more stable webRTC connection.

III Testing

8 Items to be Tested

8.1 Room Component (React + WebRTC + Socket.io)

Handles joining rooms, WebRTC connections, audio/video streams, playback synchronization, and queue updates. Testing verified connection stability, media behavior, and real-time UI updates.

8.2 Node.js Signaling Server (app.js)

Manages room creation, chat, queue updates, performer state, and WebRTC signaling. Testing focused on event handling, message validation, and consistent state propagation across clients.

8.3 FastAPI Song Service (songs_api.py)

Provides the searchable song library, database access, and YouTube/URL streaming. Testing validated API correctness, SQL queries, error handling, and media extraction reliability.

8.4 Integrated System Behavior

End-to-end testing covered room creation, multi-user video grids, synchronized playback, queue management, and chat messaging to ensure all components worked together as described in the project overview.

9 Test Specifications

9.1 Room Join & WebRTC Connection

ID#: T1 – Room Join & WebRTC Initialization

Description: Verifies that a user can join a room and establish WebRTC connections with existing participants.

Items Covered: Room Component (React), WebRTC signaling, media initialization

Requirements Addressed: Scenario 1 – Room Creation & Entry; A/V Verification

Environmental Needs: Browser with webcam/microphone; two or more clients

Intercase Dependencies: None

Test Procedures:

1. Open two browser clients.
2. Client A creates a room.
3. Client B joins using the room ID.
4. Observe video/audio streams and connection logs.

Input Specification: Valid room ID; browser media permissions enabled

Output Specification: Both clients see each other's video and audio streams

Pass/Fail Criteria: All peers connect successfully and streams appear without errors

9.2 Synchronized Video Playback

ID#: T2 – Playback Sync

Description: Ensures that play, pause, and seek actions synchronize across all clients.

Items Covered: Room Component, Socket.io events, video player

Requirements Addressed: Scenario 2 – Synchronized Playback

Environmental Needs: Two or more clients; valid song in queue

Intercase Dependencies: T1 must pass

Test Procedures:

1. Add a song to the queue.
2. Client A presses Play.
3. Client B observes playback start.
4. Client A seeks a new timestamp.
5. Client B checks for synchronized position.

Input Specification: Song selection; playback actions

Output Specification: All clients show identical playback state and timestamp

Pass/Fail Criteria: Playback remains synchronized within 0.5 seconds across clients

9.3 Queue Update Propagation

ID#: T3 – Queue Consistency

Description: Verifies that queue additions and removals propagate to all users.

Items Covered: Room Component, Node.js signaling server

Requirements Addressed: Scenario 2 – Shared Queue

Environmental Needs: Two or more clients

Intercase Dependencies: T1

Test Procedures:

1. Client A adds a song to the queue.
2. Client B observes queue updates.
3. Client A removes the song.
4. Client B observes removal.

Input Specification: Song add/remove actions

Output Specification: Queue updates appear identically on all clients

Pass/Fail Criteria: Queue state matches across all clients at all times

9.4 Song Search & API Response

ID#: T4 – Song Library Search

Description: Tests the FastAPI service for correct search results and response formatting.

Items Covered: FastAPI Song Service, SQLite database

Requirements Addressed: Scenario 2 – Song Search

Environmental Needs: Running FastAPI backend; valid database

Intercase Dependencies: None

Test Procedures:

1. Send a search query (e.g., “love”).
2. Verify returned JSON structure.
3. Confirm results match database entries.

Input Specification: Search string

Output Specification: JSON list of matching songs with correct fields

Pass/Fail Criteria: API returns correct results with no errors

9.5 YouTube Stream Extraction

ID#: T5 – Media Extraction

Description: Ensures yt-dlp successfully extracts playable audio/video streams.

Items Covered: FastAPI streaming endpoint

Requirements Addressed: Scenario 2 – Playback Support

Environmental Needs: Internet connection; yt-dlp installed

Intercase Dependencies: T4

Test Procedures:

1. Request a stream URL for a known YouTube video.
2. Verify that the returned stream plays in the browser.

Input Specification: Valid YouTube URL

Output Specification: Playable media stream with audio and video

Pass/Fail Criteria: Stream loads without errors and plays correctly

9.6 Mic/Camera Toggle Behavior

ID#: T6 – Track State Updates

Description: Verifies that mic and camera toggles propagate correctly to all peers.

Items Covered: Room Component, WebRTC track management

Requirements Addressed: Scenario 1 – A/V Controls

Environmental Needs: Two or more clients with webcams

Intercase Dependencies: T1

Test Procedures:

1. Client A toggles camera off.
2. Client B observes video disappearing.
3. Client A toggles mic off.
4. Client B observes audio mute.

Input Specification: Toggle actions

Output Specification: Track state changes reflected across all clients

Pass/Fail Criteria: All toggles update correctly with no stale states

10 Test Results

10.1 Room Join & WebRTC Connection

ID#: T1 – Room Join & WebRTC Initialization

Date(s) of Execution: April 20, 2026

Staff Conducting Tests: Jinghui Pang

Expected Results:

- Clients successfully join the same room
- WebRTC connections establish between all peers
- Video/audio streams appear for all users

Actual Results:

- Connections established successfully
- Video/audio streams displayed correctly
- No ICE or signaling errors observed

Test Status: Pass

10.2 Synchronized Video Playback

ID#: T2 – Playback Sync

Date(s) of Execution: April 21, 2026

Staff Conducting Tests: Svastik Patel

Expected Results:

- Play, pause, and seek actions synchronize across all clients
- Playback timestamps remain aligned

Actual Results:

- Play/pause synchronized correctly
- Seek actions caused minor desynchronization (~1–2 seconds) under network

delay

Test Status: Partial Pass

10.3 Queue Update Propagation

ID#: T3 – Queue Consistency

Date(s) of Execution: April 22, 2026

Staff Conducting Tests: Vinny Manara

Expected Results:

- Queue additions and removals appear instantly on all clients
- No duplicate or missing entries

Actual Results:

- Queue updates propagated correctly after server-side fix
- No inconsistencies observed

Test Status: Pass

10.4 Song Search & API Response

ID#: T4 – Song Library Search

Date(s) of Execution: April 23, 2026

Staff Conducting Tests: Svastik Patel

Expected Results:

- API returns correct JSON results for valid search queries
- Results match database entries

Actual Results:

- Correct JSON returned
- Search results matched database
- No errors encountered

Test Status: Pass

10.5 YouTube Stream Extraction

ID#: T5 – Media Extraction

Date(s) of Execution: April 24, 2026

Staff Conducting Tests: Jinghui Pang

Expected Results:

- yt-dlp extracts playable audio/video stream
- Stream loads and plays in browser

Actual Results:

- Stream extracted successfully
- Some formats initially lacked audio; resolved after format-selection fix

Test Status: Pass

10.6 Mic/Camera Toggle Behavior

ID#: T6 – Track State Updates

Date(s) of Execution: April 25, 2026

Staff Conducting Tests: Vinny Manara

Expected Results:

- Mic and camera toggles propagate correctly to all peers
- Track states update without delay

Actual Results:

- Toggles worked for small rooms
- In larger rooms (5+ users), occasional delays or stale track states occurred

Test Status: Partial Pass

11 Regression Testing

11.1 Regression Test R1 — Playback Synchronization

Original Test: T2 – Playback Sync

Reason for Re-Test: Initial test showed desynchronization during seek operations.

Date of Re-Execution: April 27, 2026

Tester: Svastik Patel

Result:

- Timestamp alignment improved after adjusting event timing
- Minor drift still present under high network latency

Status: Partial Pass

11.2 Regression Test R2 — Queue Update Consistency

Original Test: T3 – Queue Consistency

Reason for Re-Test: Server emitted duplicate or missing `queue-updated` events before fixes.

Date of Re-Execution: April 27, 2026

Tester: Vinny Manara

Result:

- Queue updates propagated correctly across all clients
- No duplicate events observed

Status: Pass

11.3 Regression Test R3 — YouTube Stream Extraction

Original Test: T5 – Media Extraction

Reason for Re-Test: Some extracted formats lacked audio before format-selection fix.

Date of Re-Execution: April 28, 2026

Tester: Jinghui Pang

Result:

- Correct audio/video formats selected
- Streams loaded consistently

Status: Pass

IV Inspection

12 Items to be Inspected

12.1 Room Component (React + WebRTC + Socket.io)

Submitted By: Svastik Patel

This file contains the core client-side logic for joining rooms, managing WebRTC peer connections, handling audio/video streams, synchronizing playback, managing the shared queue, and processing real-time events from the server. It is the most complex component in the system and requires detailed inspection for correctness,

race conditions, and state-management issues.

12.2 Node.js Signaling Server (app.js)

Submitted By: Vinny Manara

This server manages room creation, chat history, queue updates, performer state, and all Socket.io signaling for WebRTC. It also enforces constraints such as maximum room size and prevents duplicate queue entries. Inspection focused on event handling, room state consistency, and error handling.

12.3 FastAPI Song Service (songs_api.py)

Submitted By: Jinghui Pang

This service provides the searchable song library, database access, and YouTube/URL streaming proxy. Inspection focused on SQL queries, API correctness, error handling, and the yt-dlp streaming logic.

13 Inspection Procedures

13.1 Preparation

Before the formal inspection meetings, each team member independently reviewed the code submitted by the other two members. Reviewers used a shared checklist focusing on naming consistency, logic correctness, error handling, asynchronous behavior, WebRTC signaling flow, and alignment with the functional expectations defined in Scenario 1 and Scenario 2. Each reviewer documented their observations in advance so that the inspection meetings could focus on discussion rather than discovery. All preparation was coordinated through Discord, where team members shared files, clarified questions, and scheduled meeting times.

13.2 Inspection Meetings

The team conducted two formal inspection meetings using Discord voice channels and screen-sharing. Discord served as the primary communication platform throughout the project, allowing each author of the code to walk through their implementation in real time while reviewers highlighted issues, concerns, improvements, and unexpected behavior.

The first meeting focused on the Room component and the Node.js signaling server. The second meeting focused on the FastAPI song service and database import scripts.

During each meeting, reviewers presented their findings, while the author provided explanations or corrections, and the group collectively agreed on required changes. The use of Discord screen-sharing made it easy to step through the code, reproduce issues, and verify logic collaboratively.

13.3 Follow-Up

After the meetings, each author implemented the agreed fixes and pushed the updated code to the repository. Follow-up verification was performed asynchronously through Discord, where team members reviewed the updated code, confirmed that issues were resolved, and discussed any remaining concerns through messages. No additionally formal meetings were required.

14 Inspection Results

14.1 Room Component (React + WebRTC + Socket.io)

Author: Svastik Patel

Inspectors: Vinny Manara & Jinghui Pang

Date of Inspection: April 11, 2026

Findings:

- New users could not see the currently playing video, because the `nowPlaying` state was not broadcast to late joiners.
- The video reset to the beginning whenever a new user joined, caused by re-rendering the `<video>` element and re-triggering autoplay.
- Existing users could not see the camera feed of a newly joined user, while the new user could see everyone else. This was traced to ICE candidates being emitted before peer connections were fully initialized.
- Mic and camera toggles were not accurately reflected, because track states were not being updated across all peer connections.
- Queue updates were inconsistent, with some users not seeing newly added songs due to redundant or missing queue-updated events.

Resolutions:

- Added a server-side broadcast of `nowPlaying` to new users on join.
- Modified the video player to prevent reloading the source when unnecessary.
- Added a delay to ensure local media streams were ready before creating WebRTC offers.
- Updated mic/camera toggle logic to correctly enable/disable tracks and propagate changes.
- Consolidated queue update logic to ensure a single authoritative source of truth.

14.2 Node.js Signaling Server (app.js)

Author: Vinny Manara

Inspectors: Jinghui Pang & Svastik Patel

Date of Inspection: April 12, 2026

Findings:

- Queue did not accurately update for all users, because the server emitted queue-updated twice in some cases and not at all in others.
- New users did not receive the currently playing video, because the server did not send play-song to late joiners unless a performer was active.
- WebRTC messages lacked validation, causing occasional crashes when malformed ICE candidates were received.
- Room initialization logic was duplicated, increasing the risk of inconsistent state.

Resolutions:

- Removed redundant queue-updated emissions and ensured updates were sent exactly once.
- Added a dedicated broadcast of nowPlaying during the join sequence.
- Added validation checks for offer, answer, and candidate objects.
- Centralized room initialization into a helper function.

14.3 FastAPI Song Service (songs_api.py)

Author: Jinghui Pang

Inspectors: Vinny Manara & Svastik Patel

Date of Inspection: April 18, 2026

Findings:

- The song library did not load, due to an incorrect SQL LIKE parameter and a missing return statement in the /songs endpoint.
- YouTube videos could not be played, because YouTube blocked embedded playback without proper request headers and because some URLs required yt-dlp extraction.
- Some extracted video formats lacked audio, causing silent playback.
- Missing import for requests caused runtime errors in the streaming route.

Resolutions:

- Corrected SQL query formatting and fixed the missing return.
- Added a FastAPI streaming endpoint using yt-dlp to bypass YouTube embed restrictions.
- Improved format selection to ensure both audio and video tracks were present.
- Added the missing requests import and verified functionality.

V Recommendations and Conclusions

Based on the results of the testing and inspection processes, all major components of

the Virtual Karaoke system have successfully passed their functional verification, with each identified issue resolved prior to the final release.

Although the implementation meets the requirements defined in Scenario 1 and Scenario 2, further testing and improvements is advised to strengthen functionality. In particular, several issues that emerged during multi-user testing indicate areas where additional refinement would increase system reliability. The most significant remaining challenge is the video resetting whenever a new user joins, which suggests that the playback state is still too tightly coupled to component re-renders and should be decoupled through a more persistent synchronization mechanism. Additionally, mic and camera toggling becomes inconsistent when many users are present, indicating that track-state updates are not always propagated correctly across all peer connections. Playback synchronization also requires further attention, as skipping or seeking within the video can cause desynchronization across clients, especially under variable network conditions. Finally, intermittent issues such as the song library failing to load or queue updates not appearing for all users highlight the need for more robust error handling and state-management logic. Addressing these issues through extended stress testing, improved signaling reliability, and enhanced UI feedback will help ensure that the system remains stable and responsive under real-world usage.

VI Project Issues

15 Open Issues

Several issues remain unresolved at the conclusion of this implementation phase, and each represents an area of uncertainty that may significantly affect the stability and usability of the Virtual Karaoke system as it scales. The most prominent open issue is the video reset behavior when a new user joins a room. Although partial fixes were implemented, the underlying cause appears tied to how the video element re-renders and how playback state is synchronized across clients. A more robust synchronization mechanism is needed, but the exact approach has not yet been finalized.

Another open issue concerns inconsistent behavior when multiple users are present, particularly with mic and camera toggling. While the toggles work reliably in small rooms, larger rooms reveal timing issues in how track states propagate across peer connections. It is not yet clear whether this is due to WebRTC signaling delays, race conditions, or limitations in the current state-management structure.

Playback synchronization also remains an area of uncertainty. Skipping or seeking within the video can cause desynchronization across clients, especially under variable network conditions. Additional research is needed to determine whether this can be resolved purely through Socket.io events or whether a more advanced synchronization strategy, such as periodic timestamp correction, will be required.

Finally, intermittent issues such as the song library failing to load, queue updates not

appearing for all users, and YouTube playback restrictions indicate that external dependencies (e.g., YouTube request policies, network latency, browser autoplay rules) may continue to affect system behavior in unpredictable ways. These factors introduce uncertainty that cannot be fully resolved without further testing, refinement, and possibly architectural adjustments.

16 Waiting Room

The following features represent ideas and requirements that were identified during development but were not included in the current release of the Virtual Karaoke system. These items extend beyond the scope of Scenario 1 and Scenario 2 but may provide significant value in future iterations. They are recorded here to ensure that promising concepts are not lost and can be reconsidered as the system evolves.

Leaderboard for Upvotes

A persistent leaderboard that tracks which users receive the most upvotes during the session. This would introduce a competitive or social-reward element to the platform and encourage engagement.

Lobby System for Browsing and Joining Rooms

A centralized lobby where users can browse active karaoke rooms, view room details (e.g., number of participants, current song), and join public sessions. This would improve discoverability and support a more community driven experience.

Private vs. Public Room Creation

A feature allowing users to create private rooms protected by a join code, or public rooms open to all. This would give users more control over privacy and session management.

Song Preview Feature (Local-Only Playback)

A preview button that allows a user to listen to a short snippet of a song privately before adding it to the queue. This prevents accidental or unwanted song selections and improves the browsing experience.

User Statistics Dashboard

A profile-level statistics page showing metrics such as total songs sung, total time spent singing, most frequently performed songs, and number of upvotes received. This would add personalization and long-term engagement.

“Mute All” Feature During Performances

A host-controlled option to mute all participants except the active singer. This would reduce background noise and improve the quality of performances in larger rooms.

These items are not part of the current release due to time constraints and implementation complexity, but they represent high-value enhancements for future versions. Several such features, such as the lobby system, private room creation, and song preview, are “low-hanging fruit” that could be implemented with moderate effort and would significantly improve user experience. Others, such as global statistics and leaderboards, require persistent user accounts and additional backend infrastructure, making them more suitable for later development phases.

17 Ideas for Solutions

Several implementation ideas emerged during development that, while outside the scope of the current release, may be valuable for future work. These include exploring more reliable video synchronization strategies, such as server maintained timestamps or dedicated media synchronization libraries, to address playback resets and seek other related desynchronization issues. Additional ideas include using automated tests for the FastAPI and signaling server components, and containerizing the full system with Docker to streamline development and deployment. These notes are recorded here to preserve potential solution paths without turning them into requirements.

18 project Retrospective

Looking back on the project, one of the biggest challenges we faced was the amount of cramming and last-minute coding that occurred instead of following a steady, incremental development process. This led to unnecessary stress, rushed debugging, and several issues that could have been avoided with earlier planning and more consistent progress. At the start of the semester, our communication and leadership structure were also weak, which made it difficult to coordinate tasks and keep everyone aligned. These problems improved significantly once we created a shared calendar, established clearer expectations, and began using Discord more effectively—especially during the final week, when screen-sharing and walking through code together helped us quickly identify issues and understand each other’s work. Going forward, adopting a more disciplined development schedule, holding regular check-ins earlier in the semester, and maintaining the collaborative habits we developed near the end would greatly improve the process. Overall, the experience highlighted the importance of early communication, steady progress, and shared visibility into the codebase.

VI Glossary

Database: an organized collection of data storing. In this specific implementation, we handled it via SQLite.

Discord: an instant messaging and video calling platform for communication.

FastAPI: A web framework that built APIs with Python.

Karaoke: A type of interactive sing-along entertainment.

React: A JavaScript library for building user interfaces.

webRTC: An open-source project and API standard providing web browsers with real-time audio and video streaming.

yt-dlp: A command-line utility used by the FastAPI song service to extract directly playable media streams from external video URLs (like YouTube) to bypass complex embedding restrictions and licensing.

VI References / Bibliography

- [1] Robertson and Robertson, *Mastering the Requirements Process*.
- [2] Manara V., Pang J., & Patel S, *Virtual Karaoke Scenario 1 - Room Creation & Entry and Audio & Video Functionality*, Group 26, University of Illinois Chicago, 2026.
- [3] Manara V., Pang J., & Patel S, *Virtual Karaoke Scenario 2 - SoundTrack and Lyrics*, Group 26, University of Illinois Chicago, 2026.
- [4] Ahmadi M., Gepp J., Patel B., & Podapti J., *Virtual Karaoke Project Report*, Group 21, University of Illinois Chicago, 2021.
- [5] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, Ninth ed., Wiley, 2013.
- [6] J. Bell, "Underwater Archaeological Survey Report Template: A Sample Document for Generating Consistent Professional Reports," Underwater Archaeological Society of Chicago, Chicago, 2012.
- [7] M. Fowler, *UML Distilled, Third Edition*, Boston: Pearson Education, 2004.

VI Index

API, 6

Architecture, 5, 11

Database (SQLite), 6, 7

FastAPI, 6, 13

ICE Candidate, 9, 10

Inspection Results, 9

Node.js, 6

Performer, 6

Playback Synchronization, 7, 12

Queue, 5, 6, 9, 10

React, 6, 7, 9

Room Component, 7

Socket.io, 5, 6, 7, 9

Song Service, 8

Testing, 7

Track States (Mic/Camera), 9

WebRTC, 5, 6

yt-dlp, 8, 10